

DevOps FOSS para IoT Embebido: de CALMS a CI/CD con arduino-cli — Orquestación Docker, Mosquitto y 6 Jobs en AetherNet

Andrés Felipe Martínez Henao

DevOps – Electiva (Proyecto Final)

Tecnología en Desarrollo de Software (5º sem) – Ing. Sistemas y Computación (6º sem)

Universidad Tecnológica de Pereira

F.martinez5@utp.edu.co

Proyecto AetherNet – Área 2

IDs DEVOPS-01..15 – docs/DevOps/ – ci.yml 6 jobs – docker-compose 3 svc

Abstract—El programa DevOps UTP (T1 CALMS – T6 observabilidad, Talleres 1–2 y Proyecto Final) se instancia en AetherNet como orquestación 100% FOSS de un sistema IoT embebido. La infraestructura Docker Compose orquesta FastAPI 1.0.0-sprint1 + PostgreSQL 16 + Mosquitto 2.0 (bridge aethernet-net, volumen postgres_data, healthcheck pg_isready, depends_on healthy, topics aethernet/#). El broker expone 1883 TCP y 9001 WebSocket con ACLs mínimas y autenticación password_file. El pipeline GitHub Actions (.github/workflows/ci.yml v1.5.1) ejecuta 6 jobs: backend-test (ruff/mypy/pytest 21 tests), firmware-compile (matriz arduino-cli ESP32/AVR, 19252/6900 bytes), docker-build (build no-cache + curl /health 200), stats-test, android-build (plantilla), security-scan Trivy SARIF. Se documenta el caso especial CI/CD para firmware (FQBN esp32:esp32:esp32 vs arduino:avr:mega/uno), el puente MQTT→PostgreSQL (aiomqtt), el protocolo UART Gateway↔MEGA y la gestión de secretos (secrets.h/.env.example). Al 2026-09-11: DEVOPS-01..07 Done, RF nRF24L01 validado HW testing-rf-sprint1.md:32, láser v2 c7ce065, logs reales Sprint 3 (T1_docker_live.log 160 líneas, T1_mqtt_live.log 303 líneas).

Index Terms—DevOps CALMS, Docker Compose, Mosquitto ACL, GitHub Actions, arduino-cli, Trivy, FastAPI, IoT embebido

I. INTRODUCCIÓN: DEVOPS COMO PROYECTO FINAL

El PDF docs/DevOps/DEVOPS – EST.pdf define T1 Fundamentos CALMS, T2 Git, T3 Linux, T4 Contenedores, T5 CI/CD y T6 Monitoreo, con Taller 1 (Compose multi-servicio), Taller 2 (CI GitHub Actions) y Proyecto Final. AetherNet es ese Proyecto Final, con la particularidad de que el artefacto crítico corre en microcontroladores, no en servidores. RNF-1.1 (docker-compose) y RNF-1.2 (pipeline arduino-cli) son los anclajes formales; RNF-3.1 (FOSS) prohíbe Heroku/Render y similares – el despliegue es local LAN (plataformas con tier gratuito no se consideran por restricción FOSS).

Table I

MAPEO CALMS Y T2–T6 (FUENTE DOCS/DEVOPS/DEVOPS.MD §2)

Pilar	Tema	Evidencia AetherNet
C	Culture	FOSS estricto, AGENTS.md como contrato
A	Automation	6 jobs en cada push (§3)
L	Lean	Sprint 1 fundacional bloquea sprints 2–4
M	Measurement	KPIs + healthcheck PG
S	Sharing	Trazabilidad HU↔RF↔sprint↔archivo
T2	Git	main/develop, PRs disparan CI
T3	Linux	Mint + shell scripts arduino-cli
T4	Compose	3 svc + net + volúmenes (§2)
T5	CI/CD	6 jobs paralelos + Trivy
T6	Logs	Volúmenes Mosquitto + T1 logs reales

II. MAPEO T1–T6 → IMPLEMENTACIÓN

A. T1 CALMS

Culture: decisión FOSS sin Tuya Cloud (ADR-001). Automation: firmware, lint, tests y build Docker 100% automáticos. Lean: infra Sprint 1 habilita todo (atraso aquí = atraso total). Measurement: latencias <50/<10 ms, >85% ruido, healthcheck. Sharing: docs indexados para humanos y agentes.

B. T2 Git

Repo único, branches main/develop como triggers (ci.yml:3–9), PRs con convención MOV-05: ... (trazabilidad backlog.md). Branch protection: CI verde obligatorio para merge.

C. T3 Linux

Desarrollo sobre Linux Mint; pipeline instala arduino-cli vía curl, gestiona cores/libs por shell; diagnóstico con ping/nmap/tcpdump.

Table II
6 JOBS DEL PIPELINE (DOCS/DEVOPS/DEVOPS.MD §2 T5)

Job	Qué hace	Tema
backend-test	ruff+ mypy+ pytest 21	T5 pruebas
firmware-compile	arduino-cli matriz 3 fw	T4+T5 IoT
docker-build	build+ up+ curl /health	T4+T5 deploy
stats-test	pytest módulo estadístico	T5
android-build	JDK17 + gradlew (if:false)	- brecha
security-scan	Trivy SARIF → Security	T5 seguridad

D. T4 Contenedores (Taller 1)

docker-compose.yml: fastapi (build local Dockerfile), postgres:16-alpine, eclipse-mosquitto:2.0; red aethernet-net (resolución por nombre), volumen postgres_data, mounts read-only mosquitto.conf/acl.conf/init.sql, env vars \${POSTGRES_USER:-aethernet} con plantilla env.example/backend/.env.example, healthcheck pg_isready + depends_on healthy. Backend: FastAPI lifespan, CORS LAN, routers, SQLAlchemy async, Pydantic v2 (backend/app/main.py, routers/events.py:25 8 endpoints, schemas.py).

E. T5 CI/CD (Taller 2)

Pipeline ci.yml 1.5.1 (Tab. II). Caché pip y Arduino15; secrets GitHub vs .env local; badges y required checks.

F. T6 Monitoreo

Parcial: logs Mosquitto montados, docker-compose ps en CI, logs reales Sprint 3 (docs/logs/firmware_sprint3/:T1_docker_live.log Uvicorn+GET /health 200+Mosquitto 2.0.22 1883/9001, T1_mqtt_live.log aethernet/# access/event, seguridad/intrusion, rover/command|telemetry) documentados en Diario_DevOps.ipynb §4. Prometheus/Grafana fuera de alcance; heartbeat aethernet/system/status + /health degradado como MVP. Alertas de negocio (Telegram+LED) ≠ salud infra.

III. CASO ESPECIAL: CI/CD PARA FIRMWARE EMBEBIDO

arduino-cli 1.5.1 fijado (reproducibilidad), FQBN por hardware (esp32:esp32:esp32 gateway vs arduino:avr:mega/uno), instalación de librerías por firmware (ArduinoJson 6.21.3, RF24), tamaños 19252/6900 bytes. Regla: ningún push de firmware mergea sin compilar (AGENTS.md §3). Matriz + actions/cache~/.arduino15 reduce ≥60% el tiempo del segundo run.

IV. BROKER, BACKEND Y PUENTES

A. Mosquitto (DEVOPS-02)

mosquitto.conf: listener 1883 + listener 9001 protocol websockets, password_file,

acl_file, persistence true, log_dest file. Usuarios gateway/nodered/appbackend vía mosquitto_passwd; ACL: gateway rw aethernet/#, nodered r aethernet/seguridad|access/#, appbackend r aethernet/# w aethernet/app/#, allow_anonymous false.

B. Endpoints y puente MQTT→PG (DEVOPS-06/12)

GET /health (degraded si PG/MQTT down), POST /api/access-events, /sensor-events, /security-events, /rover/telemetry + GET con filtros (limit 1..200, sensor_type/event_type/session_id), validación 422, OpenAPI 8 ops. Puente backend/app/mqtt_bridge.py suscribe aethernet/access/event, seguridad/intrusion, rover/telemetry con aiomqtt, persiste en AccessEvent/SecurityEvent/SensorEvent, reconexión backoff 1→30 s. Tests: test_events.py 14 + test_health.py 7 (mock DB + integración, mock/integración).

C. UART Gateway↔MEGA (DEVOPS-13) y secretos (DEVOPS-11)

Serial2 115200, framing documentado en docs/uart-protocol.md (JSON por línea + checksum), encoder/decoder espejo. Credenciales fuera de git: firmware/gateway-esp32/secrets.h (gitignored) + secrets.h.example, CI genera secrets.h desde example.

V. RESULTADOS Y ESTADO

DEVOPS-01..05 Done, DEVOPS-06/07 Done (21 tests verdes, ruff/mypy OK), DEVOPS-08 Done (.env.example, secrets.h.example, .gitignore:219), DEVOPS-11 hecho, RF validado HW 2026-09-01 (nRF24 TX/RX L=120, isChipConnected=1), láser v2, app MQTT <50 ms. Sprint 3: joystick RF 250→249 99.6% TX, 33k telemetrías T3. Pendiente: DEVOPS-09 arranque único (scripts/up.sh), Prometheus/Grafana post-proyecto, habilitar android-build (quitar if:false, cache Gradle, subir APK artifact).

VI. DISCUSIÓN

DevOps embebido exige traducir Taller 1/2 web a constraints de hardware: compilación cruzada, secrets en flash, y latencias reales medidas con millis() para alimentar EST-05. La decisión LAN-only simplifica seguridad (sin TLS) pero traslada observabilidad a logs locales y healthchecks de negocio. La deuda Node-RED no afecta el bus: Telegram directo preserva RF-4.1.

VII. CONCLUSIONES

El stack FOSS orquestado cumple RNF-1.1/1.2 y sostiene trazabilidad completa; el pipeline de 6 jobs es la puerta de calidad que reemplaza tests unitarios embebidos inexistentes.

REFERENCES

- [1] UTP, Programa DevOps (Electiva), docs/DevOps/DEVOPS - EST.pdf.
- [2] AetherNet, Documentación DevOps, docs/DevOps/devops.md.
- [3] AetherNet, Roadmap DevOps, docs/DevOps/roadmap-devops.md.
- [4] AetherNet, Backlog DevOps DEVOPS-01..15, docs/DevOps/backlog-devops.md.
- [5] AetherNet, docker-compose.yml (3 svc, aethernet-net, healthcheck).
- [6] AetherNet, .github/workflows/ci.yml v1.5.1 (6 jobs, arduino-cli 1.5.1, Trivy).
- [7] AetherNet, backend/mosquitto/config/mosquitto.conf + acl.conf (aethernet/#).
- [8] AetherNet, backend/app/main.py + routers/events.py:25 (8 endpoints) + models.py 4 tablas.
- [9] AetherNet, docs/testing-rf-sprint1.md:32 (4/4 nRF24 validado).
- [10] AetherNet, docs/DevOps/notebook/Diario_DevOps.ipynb (10 celdas, logs T1).